Microprocessor Interfacing and Programming(MIP)

Instructor: Dr. Muhammad Akmal

https://scholar.google.com/citations?hl=en&user=aqfWdG0AAAAJ&view_op=list_works&sortby=pubdate

PIC Microcontroller



PIC originally stands for Peripheral Interface Controller.

Why MIP matters?

- **1. Core Understanding of Computer Architecture**

Students learn how a microprocessor works internally (instruction set, registers, memory, buses, control signals).

Builds intuition for how software instructions are executed at the hardware level.

- **2. Programming at the Hardware Level**

Introduces **assembly language programming**, which strengthens low-level problem-solving skills.

Helps students understand optimization, instruction cycles, and hardware-software interaction.

- **3. Interfacing with External Devices**

Teaches how microprocessors communicate with **sensors, displays, memory modules, and I/O devices**.

This is the backbone of **embedded systems** and **IoT devices**.

Why MIP matters?

- **4. Foundation for Advanced Courses**

Provides groundwork for courses like **Embedded Systems, Computer Organization, and Robotics**.

- **5. Practical Applications**

Used in real-world applications such as **automotive systems, medical devices, consumer electronics, industrial automation, and defense systems**.

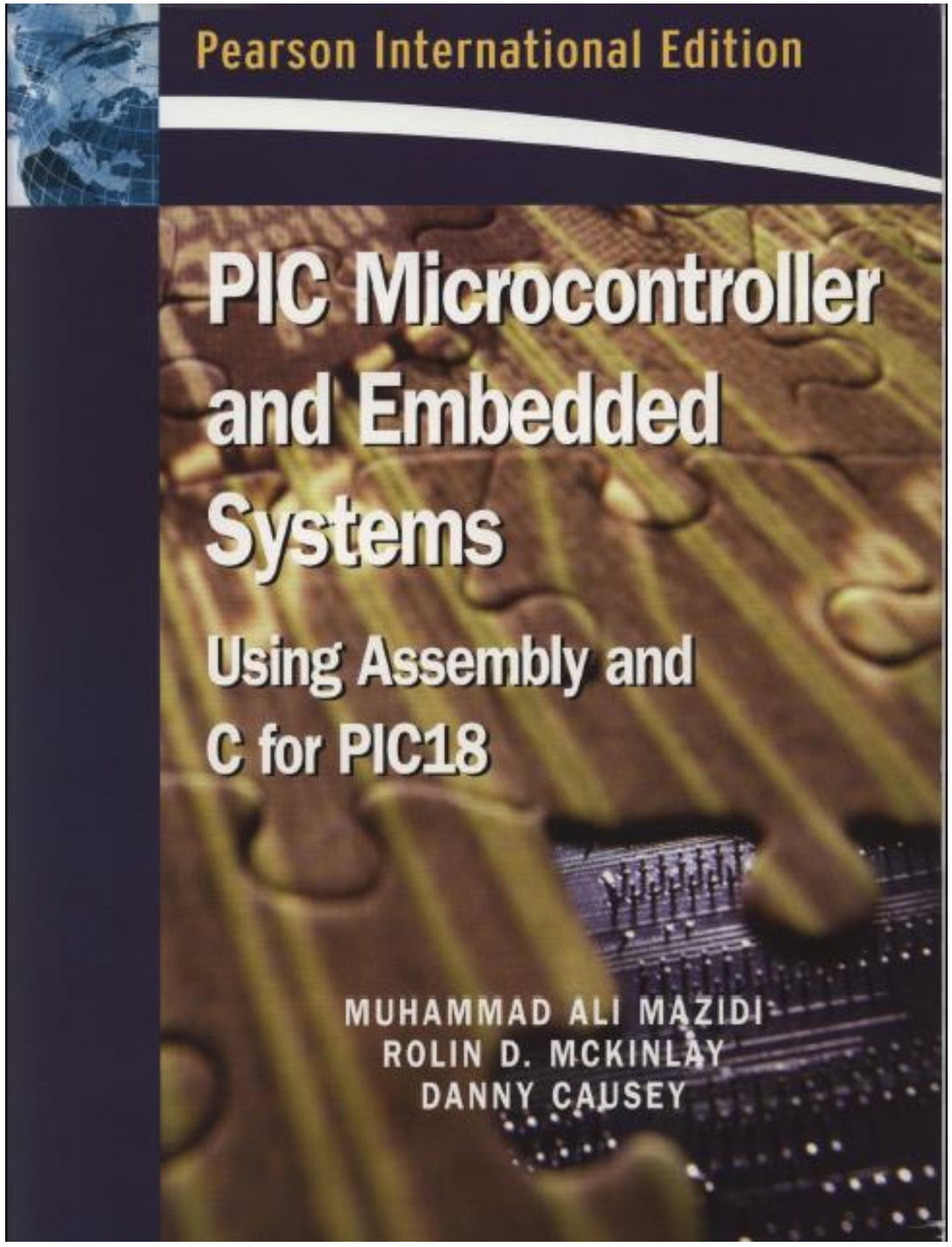
Students gain hands-on skills in building prototypes with microcontrollers and development kits.

- **6. Bridging Hardware and Software**

The course highlights how software controls hardware, which is crucial for careers in **embedded engineering, robotics, firmware development, and computer engineering**.

In short: The course is important because it teaches students how computers “think” at the smallest level and how they interact with the external world, making it the foundation for modern technology applications.




Pearson International Edition
PIC Microcontroller and Embedded Systems
Using Assembly and C for PIC18
MUHAMMAD ALI MAZIDI
ROLIN D. MCKINLAY
DANNY CAUSEY

CHAPTERS

0:	Introduction to Computing	1
1:	The PIC Microcontrollers: History and Features	23
2:	PIC Architecture & Assembly Language Programming	39
3:	Branch, Call, and Time Delay Loop	97
4:	PIC I/O Port Programming	129
5:	Arithmetic, Logic Instructions, and Programs	155
6:	Bank Switching, Table Processing, Macros, and Modules	193
7:	PIC Programming in C	251
8:	PIC18F Hardware Connection and ROM Loaders	299
9:	PIC18 Timer Programming in Assembly and C	335
10:	PIC18 Serial Port Programming in Assembly and C	387
11:	Interrupt Programming in Assembly and C	423
12:	LCD and Keyboard Interfacing	473
13:	ADC, DAC, and Sensor Interfacing	499
14:	Using Flash and EEPROM Memories for Data Storage	529
15:	CCP and ECCP Programming	569
16:	SPI Protocol and DS1306 RTC Interfacing	603
17:	Motor Control: Relay, PWM, DC, and Stepper Motors	635

INTRODUCTION

Products using microprocessors generally fall into two categories. The first category uses high-performance microprocessors such as the Pentium in applications where system performance is critical. We have an entire book dedicated to this topic, *The 80x86 IBM PC and Compatible Computers, Volumes I and II*, from Prentice Hall. In the second category of applications, performance is secondary; issues of cost, space, power, and rapid development are more critical than raw processing power. The microprocessor for this category is often called a microcontroller.

This book is for the second category of applications. The PIC18 is a widely used microcontroller. There are many reasons for this, including the existence of massive support in both software and hardware by Microchip Technology. This book is intended for use in college-level courses teaching microcontrollers and embedded systems. It not only establishes a foundation of Assembly language programming, but also provides a comprehensive treatment of PIC18 interfacing for engineering students. From this background, the design and interfacing of microcontroller-based embedded systems can be explored. This book can also be used by practicing technicians, hardware engineers, computer scientists, and hobbyists. It is an ideal source for those building stand-alone projects, or projects in which data is collected and fed into a PC for distribution on a network.

Prerequisites

Readers should have had an introductory digital course. Knowledge of Assembly language would be helpful but is not necessary. Although the book is written for those with no background in Assembly language programming, students with prior Assembly language experience will be able to gain a mastery of PIC18 architecture very rapidly and start on their projects right away. For the PIC18 C programming sections of the book, a basic knowledge of C programming is required. We use the PIC18 C compiler from Microchip Technology throughout the book. The PIC18 C compiler is compatible with MPLAB and is available for free from the Microchip website (www.microchip.com). We encourage you to use the MPLAB to simulate and run the programs in this book.

Overview

A systematic, step-by-step approach is used to cover various aspects of PIC18 C and Assembly language programming and interfacing. Many examples and sample programs are given to clarify the concepts and provide students with an opportunity to learn by doing. Review questions are provided at the end of each section to reinforce the main points of the section.

Chapter 0 covers number systems (binary, decimal, and hex), and provides an introduction to basic logic gates and computer terminology. This is designed especially for students, such as mechanical engineering students who have not taken a digital logic course or those who need to refresh their memory on these topics.

Chapter 1 discusses the history of the PIC18 and features of other PIC family members such as the PIC16. It also provides a list of various members of the PIC18 family.

Chapter 2 discusses the internal architecture of the PIC18 and explains the use of a PIC18 assembler to create ready-to-run programs. It also explores the stack and the flag register.

In Chapter 3 the topics of loop, jump, and call instructions are discussed, with many programming examples.

Chapter 4 is dedicated to the discussion of I/O ports. This allows students who are working on a project to start experimenting with PIC18 I/O interfacing and start the project as soon as possible.

Chapter 5 is dedicated to arithmetic, logic instructions, and programs.

Chapter 6 covers the PIC18 addressing modes and explains how to access the data stored in the code space of the PIC18, as well as how to do bank switching.

The C programming of the PIC18 is covered in Chapter 7. We use the PIC18 C compiler from Microchip Technology for this and other C programs of the PIC18 family throughout the book. The PIC18 C compiler is compatible with MPLAB and is available for free from the www.microchip.com website.

In Chapter 8 we discuss the hardware connection of the PIC18 chip.

Chapter 9 describes the PIC18 timers and how to use them as event counters.

Chapter 10 is dedicated to serial data communication of the PIC18 and its interfacing to the RS232. It also shows PIC18 communication with COM ports of the x86 IBM PC and compatible computers.

Chapter 11 provides a detailed discussion of PIC18 interrupts with many examples on how to write interrupt handler programs.

Chapter 12 shows PIC18 interfacing with real-world devices such as LCDs and keyboards.

Chapter 13 shows PIC18 interfacing with real-world devices such as DAC chips, ADC chips, and sensors.

In Chapter 14 we cover how to use PIC18 Flash and EEPROM memories for data storage.

Chapter 15 covers the CCP and ECCP modules inside the PIC18 and shows how they are used.

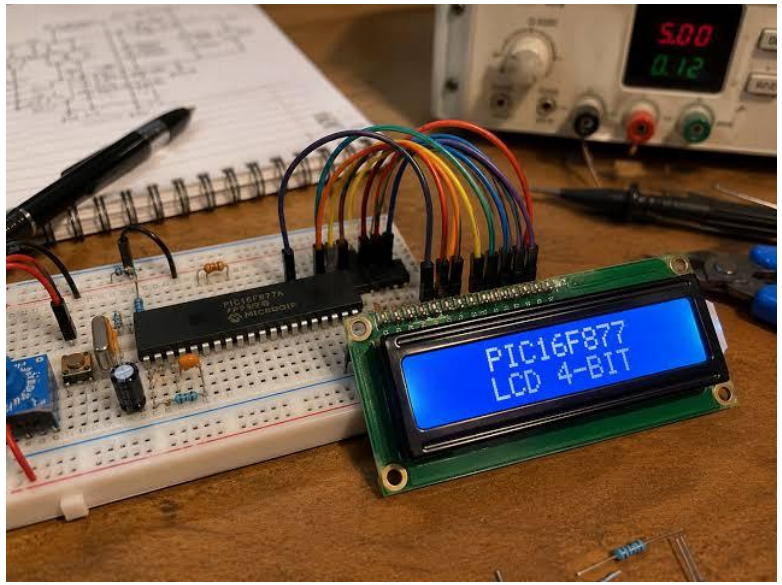
Chapter 16 shows how to connect and program the DS1306 real-time clock chip using the SPI bus protocol.

Finally, Chapter 17 shows basic interfacing to relays, optoisolators, and motors.

CHAPTER 0

INTRODUCTION TO COMPUTING

To understand the software and hardware of a microcontroller-based system, one must first master some very basic concepts underlying computer design. In this chapter (which in the tradition of digital computers is called Chapter 0), the fundamentals of numbering and coding systems are presented. After an introduction to logic gates, an overview of the workings inside the computer is given. Finally, in the last section we give a brief history of CPU architecture. Although some readers may have an adequate background in many of the topics of this chapter, it is recommended that the material be scanned, however briefly.



Decimal and binary number systems

Converting from decimal to binary

One method of converting from decimal to binary is to divide the decimal number by 2 repeatedly, keeping track of the remainders. This process continues until the quotient becomes zero. The remainders are then written in reverse order to obtain the binary number. This is demonstrated in Example 0-1.

Example 0-1

Convert 25_{10} to binary.

Solution:

	Quotient	*Remainder*	
25/2 =	12	1	LSB (least significant bit)
12/2 =	6	0	
6/2 =	3	0	
3/2 =	1	1	
1/2 =	0	1	MSB (most significant bit)

Therefore, $25_{10} = 11001_2$.

Converting from binary to decimal

110101_2 =				***Decimal***
1×2^0	=	1×1	=	1
0×2^1	=	0×2	=	0
1×2^2	=	1×4	=	4
0×2^3	=	0×8	=	0
1×2^4	=	1×16	=	16
1×2^5	=	1×32	=	32
				53

Example 0-2

Convert 11001_2 to decimal.

Knowing the weight associated with each binary bit position allows one to convert a decimal number to binary directly instead of going through the process of repeated division. This is shown in Example 0-3.

Example 0-3

Use the concept of weight to convert 39_{10} to binary.

Example 0-2

Convert 11001_2 to decimal.

Solution:

Weight:	16	8	4	2	1
Digits:	1	1	0	0	1
Sum:	16 +	8 +	0 +	0 +	$1 = 25_{10}$

Knowing the weight associated with each binary bit position allows one to convert a decimal number to binary directly instead of going through the process of repeated division. This is shown in Example 0-3.

Example 0-3

Use the concept of weight to convert 39_{10} to binary.

Solution:

Weight:	32	16	8	4	2	1
	1	0	0	1	1	1
	32 +	0 +	0 +	4 +	2 +	$1 = 39$

Therefore, $39_{10} = 100111_2$.

Hexadecimal system

Table 0-1: Base 16 Number System

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Converting between binary and hex

Example 0-4

Represent binary 100111110101 in hex.

Solution:
First the number is grouped into sets of 4 bits: 1001 1111 0101.
Then each group of 4 bits is replaced with its hex equivalent:

1001	1111	0101
9	F	5

Therefore, 100111110101_2 = 9F5 hexadecimal.

Example 0-5

Convert hex 29B to binary.

Solution:

	2	9	B
=	0010	1001	1011

Dropping the leading zeros gives 1010011011.

Converting from decimal to hex

Example 0-6

(a) Convert 45_{10} to hex.

32	16	8	4	2	1	First, convert to binary.
1	0	1	1	0	1	32 + 8 + 4 + 1 = 45

$45_{10} = 0010\ 1101_2$ = 2D hex

(b) Convert 629_{10} to hex.

(c) Convert 1714_{10} to hex.

Converting from decimal to hex

(b) Convert 629_{10} to hex.

512	256	128	64	32	16	8	4	2	1
1	0	0	1	1	1	0	1	0	1

$629_{10} = (512 + 64 + 32 + 16 + 4 + 1) = 0010\ 0111\ 0101_2 = 275$ hex

(c) Convert 1714_{10} to hex.

1024	512	256	128	64	32	16	8	4	2	1
1	1	0	1	0	1	1	0	0	1	0

$1714_{10} = (1024 + 512 + 128 + 32 + 16 + 2) = 0110\ 1011\ 0010_2 = 6B2$ hex

Converting from hex to decimal

Example 0-7

Convert the following hexadecimal numbers to decimal.

(a) $6B2_{16} = 0110\ 1011\ 0010_2$

1024	512	256	128	64	32	16	8	4	2	1
1	1	0	1	0	1	1	0	0	1	0

$1024 + 512 + 128 + 32 + 16 + 2 = 1714_{10}$

(b) $9F2D_{16} = (?)$

Converting from hex to decimal

(b) $9F2D_{16} = 1001\ 1111\ 0010\ 1101_2$

32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
1	0	0	1	1	1	1	1	0	0	1	0	1	1	0	1

$32768 + 4096 + 2048 + 1024 + 512 + 256 + 32 + 8 + 4 + 1 = 40{,}749_{10}$

Table 0-2: Counting in Bases

Decimal	Binary	Hex
0	00000	0
1	00001	1
2	00010	2
3	00011	3
4	00100	4
5	00101	5
6	00110	6
7	00111	7
8	01000	8
9	01001	9
10	01010	A
11	01011	B
12	01100	C
13	01101	D
14	01110	E
15	01111	F
16	10000	10

Counting in bases 10, 2, and 16

To show the relationship between all three bases, in Table 0-2 we show the sequence of numbers from 0 to 31 in decimal, along with the equivalent binary and hex numbers. Notice in each base that when one more is added to the highest digit, that digit becomes zero and a 1 is carried to the next-highest digit position. For example, in decimal, 9 + 1 = 0 with a carry to the next-highest position. In binary, 1 + 1 = 0 with a carry; similarly, in hex, F + 1 = 0 with a carry.

Table 0-3: Binary Addition

A + B	Carry	Sum
0 + 0	0	0
0 + 1	0	1
1 + 0	0	1
1 + 1	1	0

Addition of binary and hex numbers

The addition of binary numbers is a very straightforward process. Table 0-3 shows the addition of two bits. The discussion of subtraction of binary numbers is bypassed since all computers use the addition process to implement subtraction. Although computers have adder circuitry, there is no separate circuitry for subtractors. Instead, adders are used in conjunction with *2's complement* circuitry to perform subtraction. In other words, to implement "$x - y$", the computer takes the 2's complement of y and adds it to x. The concept of 2's complement is reviewed next. Example 0-8 shows the addition of binary numbers.

Table 0-3: Binary Addition

A + B	Carry	Sum
0 + 0	0	0
0 + 1	0	1
1 + 0	0	1
1 + 1	1	0

Example 0-8

Add the following binary numbers. Check against their decimal equivalents.

Solution:

	Binary	*Decimal*
	1101	13
+	1001	9
	10110	22

2's complement

To get the 2's complement of a binary number, invert all the bits and then add 1 to the result. Inverting the bits is simply a matter of changing all 0s to 1s and 1s to 0s. This is called the *1's complement*. See Example 0-9.

Example 0-9

Take the 2's complement of 10011101.

Solution:

	10011101	binary number
	01100010	1's complement
+	1	
	01100011	2's complement

Subtraction using 2's complement

$4 - 2 = (?)$

$4 + (-2)$

Take two's complement of 2:

$(2)_{10} \Rightarrow (010)_2$

101 ⇐ 1's complement

$+ \quad 1$

$(110)_2 \Rightarrow (-2)_{10}$

Now add $(110)_2$ in $(100)_2$

$$\begin{array}{r} 100 \\ +\ 110 \\ \hline (1)010 \end{array}$$

Neglect this

$(010)_2 = (2)_{10}$

In short:

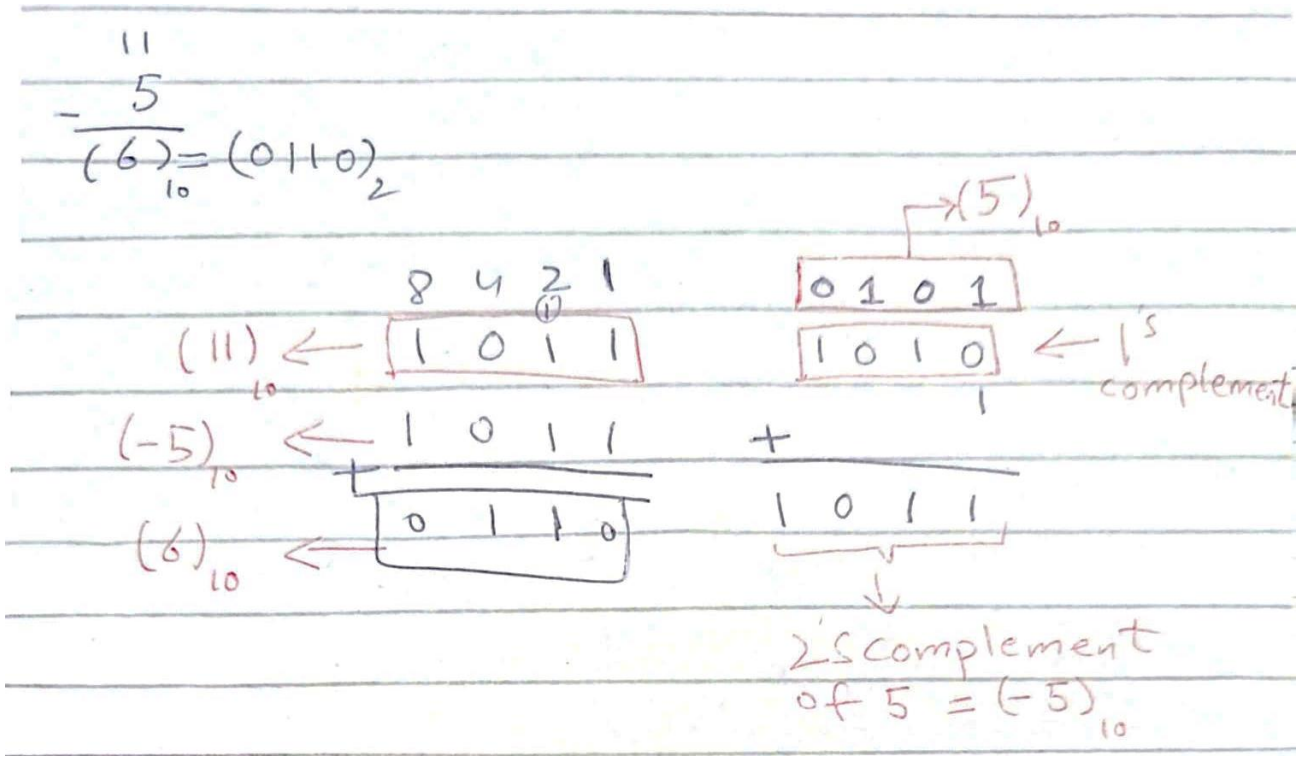
$$\begin{array}{ccc} 4 & 4 & 100 \\ -2 = & +(-2) = & 110 \\ \hline (2)_{10} & (2)_{10} & (010)_2 \end{array}$$

Validate following subtraction using 2’s complement

$$(11)_{10} - (5)_{10} = (6)_{10}$$

Validate following subtraction using 2's complement

$$(11)_{10} - (5)_{10} = (6)_{10}$$



SECTION 0.2: DIGITAL PRIMER

This section gives an overview of digital logic and design. First, we cover binary logic operations, then we show gates that perform these functions. Next, logic gates are put together to form simple digital circuits. Finally, we cover some logic devices commonly found in microcontroller interfacing.

Binary logic

As mentioned earlier, computers use the binary number system because the two voltage levels can be represented as the two digits 0 and 1. Signals in digital electronics have two distinct voltage levels. For example, a system may define 0 V as logic 0 and +5 V as logic 1. Figure 0-2 shows this system with the built-in tolerances for variations in the voltage.

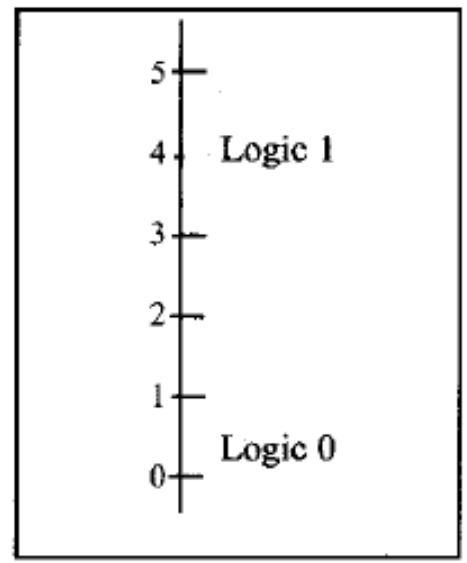


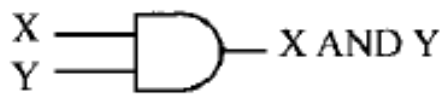
Figure 0-2. Binary Signals

Logic gates

AND gate

Logical AND Function

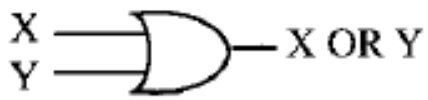
Inputs	Output
X Y	X AND Y
0 0	0
0 1	0
1 0	0
1 1	1



OR gate

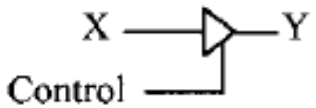
Logical OR Function

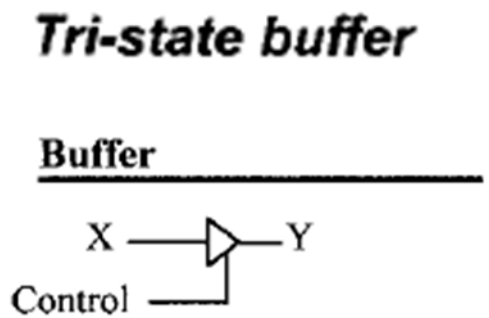
Inputs	Output
X Y	X OR Y
0 0	0
0 1	1
1 0	1
1 1	1



Tri-state buffer

Buffer





A tri-state buffer adds an **Enable (EN)** input:

Enable	Input	Output
1	0	0
1	1	1
0	0 or 1	Hi-Z

When **EN = 1**, the buffer passes the input to the output.

When **EN = 0**, the output becomes **High-Z**.

2. What is High-Z?

High impedance means that the buffer essentially **disconnects its output from the circuit**.

It is not the same as logic 0.

Think of it as an electrical **open circuit**:

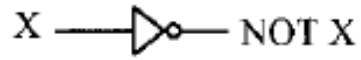
```
              EN = 1
Input ──────► [Tri-State] ──────► Bus
                  │
              output = 0/1


              EN = 0
Input ──────► [Tri-State] ──────► Bus
                                   │
                                 Hi-Z
                              (disconnected)
```

Inverter

Logical Inverter

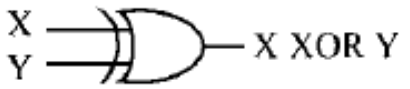
Input	Output
X	NOT X
0	1
1	0



XOR gate

Logical XOR Function

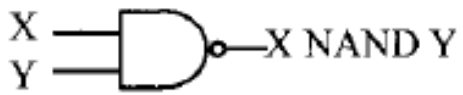
Inputs	Output
X Y	X XOR Y
0 0	0
0 1	1
1 0	1
1 1	0



NAND and NOR gates

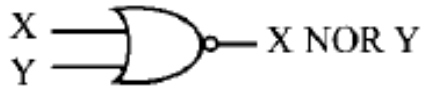
Logical NAND Function

Inputs	Output
X Y	X NAND Y
0 0	1
0 1	1
1 0	1
1 1	0

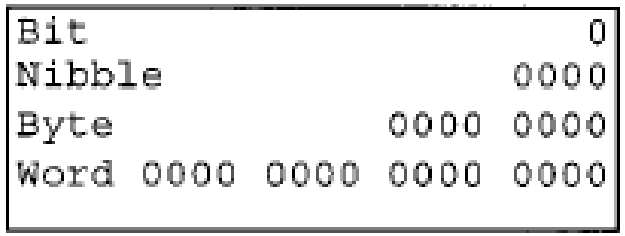


Logical NOR Function

Inputs	Output
X Y	X NOR Y
0 0	1
0 1	0
1 0	0
1 1	0



Some important terminology



A *kilobyte* is 2^{10} bytes, which is 1024 bytes. The abbreviation K is often used to represent kilobytes. For example, some floppy disks hold 356K of data. A *megabyte*, or *meg* as some call it, is 2^{20} bytes. That is a little over 1 million bytes; it is exactly 1,048,576 bytes. Moving rapidly up the scale in size, a *gigabyte* is 2^{30} bytes (over 1 billion), and a *terabyte* is 2^{40} bytes (over 1 trillion). As an example of how some of these terms are used, suppose that a given computer has 16 megabytes of memory. That would be 16×2^{20}, or $2^4 \times 2^{20}$, which is 2^{24}. Therefore 16 megabytes is 2^{24} bytes.

Two types of memory commonly used in microcomputers are *RAM*, which stands for "random access memory" (sometimes called *read/write memory*), and *ROM*, which stands for "read-only memory." RAM is used by the computer for temporary storage of programs that it is running. That data is lost when the computer is turned off. For this reason, RAM is sometimes called *volatile memory*. ROM contains programs and information essential to operation of the computer. The information in ROM is permanent, cannot be changed by the user, and is not lost when the power is turned off. Therefore, it is called *nonvolatile memory*.

Internal organization of computers

The internal working of every computer can be broken down into three parts: CPU (central processing unit), memory, and I/O (input/output) devices (see Figure 0-9). The function of the CPU is to execute (process) information stored in memory. The function of I/O devices such as the keyboard and video monitor is to provide a means of communicating with the CPU. The CPU is connected to memory and I/O through strips of wire called a *bus*. The bus inside a computer carries information from place to place just as a street bus carries people from place to place. In every computer there are three types of buses: address bus, data bus, and control bus.

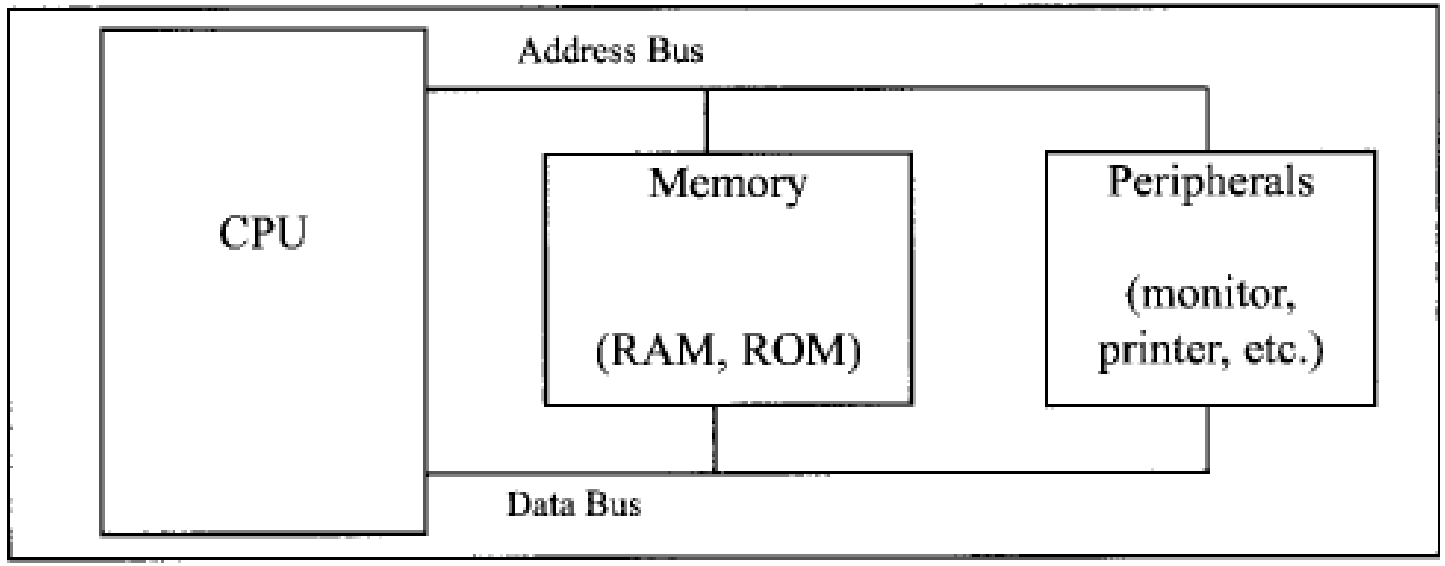


Figure 0-9. Inside the Computer

For a device (memory or I/O) to be recognized by the CPU, it must be assigned an address. The address assigned to a given device must be unique; no two devices are allowed to have the same address. The CPU puts the address (in binary, of course) on the address bus, and the decoding circuitry finds the device. Then the CPU uses the data bus either to get data from that device or to send data to it. The control buses are used to provide read or write signals to the device to indicate if the CPU is asking for information or sending information. Of the three buses, the address bus and data bus determine the capability of a given CPU.

More about the data bus

Because data buses are used to carry information in and out of a CPU, the more data buses available, the better the CPU. If one thinks of data buses as highway lanes, it is clear that more lanes provide a better pathway between the CPU and its external devices (such as printers, RAM, ROM, etc.; see Figure 0-10). By the same token, that increase in the number of lanes increases the cost of construction. More data buses mean a more expensive CPU and computer. The average size of data buses in CPUs varies between 8 and 64. Early personal computers such as Apple 2 used an 8-bit data bus, while supercomputers such as Cray use a 64-bit data bus. Data buses are bidirectional, because the CPU must use them either to receive or to send data. The processing power of a computer is related to the size of its buses, because an 8-bit bus can send out 1 byte a time, but a 16-bit bus can send out 2 bytes at a time, which is twice as fast.